

README

Finex DSL

November 27, 2025

1 Introduction

Finex is a statically-typed Domain Specific Language (DSL) designed for financial modeling. The compiler is built using Flex and Bison and transpiles to C++ for performance. Finex provides simple unified framework and also supports the financial overflow which includes risk analysis and portfolio management. It supports fraud detection, banking operations, and further integrates statistical methods and machine learning models. To conclude, Finex supports as a both computational layer and an abstraction layer.

2 Compiler Building

- **Requirements:** g++, flex, bison
- **Command:** make
- **Command to clean build files:** make clean
- **Build Pipeline:**
 - **Lexical Analysis:** lexer.l is compiled by FLex to generate tokens.
 - **Parsing:** parser.y is processed by Bison to generate the grammar logic
 - **AST Generation:** The parser constructs a tree of C++ classes inheriting from ASTNode
 - **Semantic Analysis:** A Visitor Pattern implementation (with 2 passes) traverses the AST to build the symbol table and verify types
 - **Linking:** All components are linked into the finex_parser executable
- **Command to run:** ./finex test_input.fx

3 Project Directory Structure

```
|-- Makefile                # Build script (needs to be updated for src/ and include/ paths)
|
|-- Header_Files/          # Header files (.hpp)
|   |-- ast.hpp             # AST node definitions
|   |-- semantic_analyzer.hpp # Semantic analysis & type checking declarations
|   +-- symbol_table.hpp    # Symbol table class declarations
|
|-- Source_Code/           # Source files (.cpp, .l, .y)
|   |-- ast.cpp             # AST implementation
|   |-- lexer.l             # Flex lexer definition
|   |-- parser.y            # Bison parser grammar & main()
|   |-- semantic_analyzer.cpp # Semantic analysis logic
|   +-- symbol_table.cpp    # Symbol table implementation
|
+-- Test_Cases/            # Test programs (.fx)
    |-- phase1_basic/       # Basic AST & Parsing tests
```

```

| |-- test_input1.fx
| |-- test_input2.fx
| +-- test_input3.fx
|
|-- phase2_syntab/          # Symbol Table verification
| |-- test_input4.fx
| |-- test_input5.fx
| +-- test_input6.fx
|
|-- phase3_errors/         # Syntax Error handling tests
| |-- test_input7.fx
| |-- test_input8.fx
| |-- test_input9.fx
| |-- test_input10.fx
| |-- test_input11.fx
| |-- test_input12.fx
| +-- test_input13.fx
|
+-- phase4_semantics/      # Type checking & Semantic tests
| |-- test_input14.fx
| |-- test_input15.fx
| |-- test_input16.fx
| +-- test_input17.fx

```

4 Sample AST Output

Input Source	Program
Client Logic	AST & Symbol Table
<pre> ## 1. Record Definition record Client { string name; int id; }; ## 2. Function Definition int calculateBonus(int score) { int multiplier = 10; return score * multiplier; } void main() { ## 3. Variable Declarations int score = 50; real rate = 2.5; string status = "Active"; ## 5. Record Instantiation Client c; ## 6. Control Flow if (score > 10) { score = calculateBonus(score); } ## 7. Loop FIXED: Declared 'i' outside int i; for (i = 0; i < 5; i += 1) { score = score + 1; } </pre>	<pre> === AST Output === Program: RecordDefinition: Client Declaration: Type: string Declarator: name Declaration: Type: int Declarator: id Declaration: Type: Client FunctionDefinition: Type: int Declarator: calculateBonus [function] CompoundStatement: Declaration: Type: int Declarator: multiplier Initializer: Literal[int]: 10 JumpStatement: return BinaryOp: * Identifier: score Identifier: multiplier FunctionDefinition: Type: void Declarator: main [function] CompoundStatement: Declaration: Type: int Declarator: score </pre>

}

```
        Initializer:
            Literal[int]: 50
Declaration:
    Type: real
    Declarator: rate
        Initializer:
            Literal[real]: 2.5
Declaration:
    Type: string
    Declarator: status
        Initializer:
            Literal[string]: "Active"
Declaration:
    Type: Client
    Declarator: c
IfStatement:
    Condition:
        BinaryOp: >
            Identifier: score
            Literal[int]: 10
    Then:
        CompoundStatement:
            ExpressionStatement:
                Assignment: =
                    Identifier: score
                    FunctionCall:
                        Identifier: calculateBonus
                        Identifier: score
Declaration:
    Type: int
    Declarator: i
ForStatement[C-style]:
    Assignment: =
        Identifier: i
        Literal[int]: 0
    BinaryOp: <
        Identifier: i
        Literal[int]: 5
    Assignment: +=
        Identifier: i
        Literal[int]: 1
    CompoundStatement:
        ExpressionStatement:
            Assignment: =
                Identifier: score
            BinaryOp: +
                Identifier: score
                Literal[int]: 1

=== Semantic Analysis ===
[SymbolTable] Initializing Global Scope (ID:
0)
Running Pass 1: Declarations...
[SymbolTable] Inserting 'Client' (Type) into
Scope ID: 0 [L:5, C:2] ... Success.
[SymbolTable] Inserting 'calculateBonus' (
Func) into Scope ID: 0 [L:11, C:2] ...
Success.
[SymbolTable] Inserting 'main' (Func) into
Scope ID: 0 [L:32, C:2] ... Success.
Running Pass 2: Checks...
[SymbolTable] Resetting cursor to Global
Scope
```

```

[SymbolTable] Entering new Scope ID: 1 (
  Parent ID: 0)
[SymbolTable] Inserting 'score' (Param) into
  Scope ID: 1 [L:8, C:30] ... Success.
[SymbolTable] Inserting 'multiplier' (Var)
  into Scope ID: 1 [L:9, C:25] ... Success.
[SymbolTable] Exiting Scope ID: 1 ->
  Returning to Scope ID: 0
[SymbolTable] Entering new Scope ID: 2 (
  Parent ID: 0)
[SymbolTable] Inserting 'score' (Var) into
  Scope ID: 2 [L:15, C:20] ... Success.
[SymbolTable] Inserting 'rate' (Var) into
  Scope ID: 2 [L:16, C:21] ... Success.
[SymbolTable] Inserting 'status' (Var) into
  Scope ID: 2 [L:17, C:30] ... Success.
[SymbolTable] Inserting 'c' (Var) into Scope
  ID: 2 [L:20, C:14] ... Success.
[SymbolTable] Entering new Scope ID: 3 (
  Parent ID: 2)
[SymbolTable] Exiting Scope ID: 3 ->
  Returning to Scope ID: 2
[SymbolTable] Inserting 'i' (Var) into Scope
  ID: 2 [L:28, C:11] ... Success.
[SymbolTable] Entering new Scope ID: 4 (
  Parent ID: 2)
[SymbolTable] Entering new Scope ID: 5 (
  Parent ID: 4)
[SymbolTable] Exiting Scope ID: 5 ->
  Returning to Scope ID: 4
[SymbolTable] Exiting Scope ID: 4 ->
  Returning to Scope ID: 2
[SymbolTable] Exiting Scope ID: 2 ->
  Returning to Scope ID: 0

=== SYMBOL TABLE DUMP ===
Scope 0 (Global):
- main [Func] @32:2 type: primitive
- calculateBonus [Func] @11:2 type:
  primitive
- Client [Type] @5:2
Scope 1:
- multiplier [Var] @9:25 type: primitive
- score [Param] @8:30 type: primitive
Scope 2:
- i [Var] @28:11 type: primitive
- c [Var] @20:14 type: Client
- status [Var] @17:30 type: primitive
- rate [Var] @16:21 type: primitive
- score [Var] @15:20 type: primitive
Scope 3:
Scope 4:
Scope 5:
=====

```

Semantic Errors Records

```
## =====
## Finex Function & Record Semantic Test
## =====

## [Fix 1] Added semicolon at the end of
    record definition
record Point {
    int x;
    int y;
};

int add_score(int base, int bonus) {
    return base + bonus;
}

## [Semantic Error 1] Return Type Mismatch (
    String vs Int)
int bad_return() {
    return "Wrong Type";
}

int main() {
    Point p;

    ## [Fix 2] Used '->' instead of '.'
    p->x = 10;
    p->y = 20;

    ## [Semantic Error 2] Member Access:
    Member 'z' does not exist
    p->z = 30;

    ## 1. Valid Function Call
    int s = add_score(100, 50);

    ## [Semantic Error 3] Argument Count
    Mismatch
    int f = add_score(100);

    ## [Semantic Error 4] Argument Type
    Mismatch
    int g = add_score(100, "Bonus");

    return 0;
}
```

Error Logging AST

```
=== AST Output ===
Program:
  RecordDefinition: Point
    Declaration:
      Type: int
      Declarator: x
    Declaration:
      Type: int
      Declarator: y
  Declaration:
    Type: Point
  FunctionDefinition:
    Type: int
    Declarator: add_score [function]
    CompoundStatement:
      JumpStatement: return
      BinaryOp: +
        Identifier: base
        Identifier: bonus
  FunctionDefinition:
    Type: int
    Declarator: bad_return [function]
    CompoundStatement:
      JumpStatement: return
        Literal[string]: "Wrong Type"
  FunctionDefinition:
    Type: int
    Declarator: main [function]
    CompoundStatement:
      Declaration:
        Type: Point
        Declarator: p
      ExpressionStatement:
        Assignment: =
          MemberAccess[->]: x
            Identifier: p
          Literal[int]: 10
      ExpressionStatement:
        Assignment: =
          MemberAccess[->]: y
            Identifier: p
          Literal[int]: 20
      ExpressionStatement:
        Assignment: =
          MemberAccess[->]: z
            Identifier: p
          Literal[int]: 30
      Declaration:
        Type: int
        Declarator: s
        Initializer:
          FunctionCall:
            Identifier: add_score
            Literal[int]: 100
            Literal[int]: 50
      Declaration:
        Type: int
        Declarator: f
        Initializer:
          FunctionCall:
            Identifier: add_score
```

```

        Literal[int]: 100
Declaration:
  Type: int
  Declarator: g
  Initializer:
    FunctionCall:
      Identifier: add_score
      Literal[int]: 100
      Literal[string]: "Bonus"
  JumpStatement: return
  Literal[int]: 0

=== Semantic Analysis ===
[SymbolTable] Initializing Global Scope (ID:
0)
Running Pass 1: Declarations...
[SymbolTable] Inserting 'Point' (Type) into
  Scope ID: 0 [L:9, C:2] ... Success.
[SymbolTable] Inserting 'add_score' (Func)
  into Scope ID: 0 [L:13, C:2] ... Success.
[SymbolTable] Inserting 'bad_return' (Func)
  into Scope ID: 0 [L:18, C:2] ... Success.
[SymbolTable] Inserting 'main' (Func) into
  Scope ID: 0 [L:40, C:2] ... Success.
Running Pass 2: Checks...
[SymbolTable] Resetting cursor to Global
  Scope
[SymbolTable] Entering new Scope ID: 1 (
  Parent ID: 0)
[SymbolTable] Inserting 'base' (Param) into
  Scope ID: 1 [L:11, C:24] ... Success.
[SymbolTable] Inserting 'bonus' (Param) into
  Scope ID: 1 [L:11, C:35] ... Success.
[SymbolTable] Exiting Scope ID: 1 ->
  Returning to Scope ID: 0
[SymbolTable] Entering new Scope ID: 2 (
  Parent ID: 0)
Semantic Error at 17:25: Return type mismatch
  . Expected int, got string
[SymbolTable] Exiting Scope ID: 2 ->
  Returning to Scope ID: 0
[SymbolTable] Entering new Scope ID: 3 (
  Parent ID: 0)
[SymbolTable] Inserting 'p' (Var) into Scope
  ID: 3 [L:21, C:13] ... Success.
Semantic Error at 28:9: Record 'Point' has no
  member named 'z'
Semantic Error at 28:15: Cannot assign value
  of type int to variable of type void
[SymbolTable] Inserting 's' (Var) into Scope
  ID: 3 [L:31, C:32] ... Success.
Semantic Error at 34:27: Incorrect argument
  count for function 'add_score'. Expected
  2, got 1
[SymbolTable] Inserting 'f' (Var) into Scope
  ID: 3 [L:34, C:28] ... Success.
Semantic Error at 37:35: Type mismatch for
  argument 2 in call to 'add_score'.
  Expected int, got string
[SymbolTable] Inserting 'g' (Var) into Scope
  ID: 3 [L:37, C:37] ... Success.
[SymbolTable] Exiting Scope ID: 3 ->
  Returning to Scope ID: 0

```

```

=== SYMBOL TABLE DUMP ===
Scope 0 (Global):
- main [Func] @40:2 type: primitive
- bad_return [Func] @18:2 type: primitive
- add_score [Func] @13:2 type: primitive
- Point [Type] @9:2
Scope 1:
- bonus [Param] @11:35 type: primitive
- base [Param] @11:24 type: primitive
Scope 2:
Scope 3:
- f [Var] @34:28 type: primitive
- g [Var] @37:37 type: primitive
- s [Var] @31:32 type: primitive
- p [Var] @21:13 type: Point

```

5 Current Status

- **Frontend** (Completed): Lexer, Parser, AST construction, Symbol table and Semantic Analysis (Type checking and Symbol Table) are fully functional.
- **Backend** (In Progress): Currently implementing the Code Generation phase to transpile the AST into executable C++ code.